Infrastructure: MCP Relay

Status: Draft v0.1
Last Updated: 2026-04-08

Contents

1. Overview
2. Infrastructure Requirements
3. Provider Options
4. Recommended Stack
5. Phased Deployment
6. Cost Considerations

---

1. Overview

This document defines the infrastructure services required to take MCP Relay from local development through to a team-accessible deployed prototype. Each requirement is described **service-agnostically** first, then mapped to provider options. For an overview of what MCP Relay is and what it aims to achieve, see the Architecture document.

Phase 2 is the critical path — company access requests to enable team-accessible deployment for prototype.

Deployment Phases

```
Phase 1 (current)                Phase 2                        Phase 3
─────────────────                ───────────────────            ─────────────────
Local MVP development            Company access & deployment    Scale & harden
stdio transport                  Streamable HTTP                Streamable HTTP
.env file auth                   Secrets manager + OAuth 2.1    OAuth 2.1 + RBAC
Single developer                 Select teams testing prototype Multi-team rollout
No hosting needed                Container hosting provisioned  HA + monitoring
No company deps                  ⚠ Blocked without access requests Production-grade
```

- **Phase 1:** Build the MVP locally; all tools functional via stdio. Audience: developer(s) building the server
- **Phase 2:** Deploy prototype for team testing; requires enterprise infrastructure access. Audience: select teams evaluating the server
- **Phase 3:** Harden, scale, and roll out to wider organisation. Audience: multiple teams in production use

---

2. Infrastructure Requirements

Each requirement is categorised by when it becomes necessary.

2.1 Compute

What: A runtime environment to host the Node.js MCP server process.

- **Runtime:** Node.js 22+ (LTS)
- **Workload type:** Long-running HTTP server (Streamable HTTP), multiple concurrent sessions
- **Resource profile:** Stateless, low CPU, low memory — thin proxy layer over external APIs
- **Availability:** Business hours initially; evolve to high availability as adoption grows

Phase needed: Phase 2 — request access to container hosting service (e.g. ECS Fargate, Cloud Run)

2.2 Secrets Management

What: Secure storage and injection of API credentials (Jira tokens, S3 keys, OAuth client secrets).

- **Secrets count:** ~10 initially (Jira, S3, OAuth), growing with each tool integration
- **Access pattern:** Read at startup + runtime; must support rotation without redeployment
- **Audit:** Log who accessed which secret and when

Phase needed: Phase 2 — request access to secrets management service (e.g. AWS Secrets Manager, Key Vault)

2.3 Object Storage

What: Storage for per-project configuration files (brand guidelines JSON). Already architected as S3-compatible in Phase 1.

- **Content type:** JSON configuration files (small, <1MB)
- **Access pattern:** Read-heavy, infrequent writes (config updates)
- **Structure:** `{bucket}/{projectId}/guidelines.json`
- **Versioning:** Desirable for config rollback

Phase needed: Phase 1 (brand guidelines tool) — request S3 bucket or equivalent storage access

2.4 Reverse Proxy & Networking

What: A reverse proxy sits in front of the MCP server, handling HTTPS termination, TLS certificates, and request forwarding. The MCP client connects to the proxy over HTTPS; the proxy forwards to the Node.js app over plain HTTP internally.

```
MCP Client (work laptop)
        │
        ▼ HTTPS (port 443)
┌──────────────────┐
│  Reverse Proxy   │  ← Public-facing, handles TLS
│  (e.g. Caddy)    │
└──────────────────┘
        │ HTTP (internal only)
        ▼
┌──────────────────┐
│  MCP Relay       │  ← Node.js server, port 3000
│  (container)     │
└──────────────────┘
```

- **TLS:** Automatic certificate provisioning and renewal (e.g. Caddy + Let's Encrypt)

- **SSE support:** Required — Streamable HTTP uses Server-Sent Events
- **Load balancing:** Not needed for prototype (single instance); defer to Phase 3
- **Zscaler:** Company uses Zscaler proxy (no VPN); currently works without whitelisting — revisit if blocked

Security considerations:

- Proxy must enforce TLS 1.2+ with strong ciphers (Caddy does this by default)
- Internal traffic (proxy → app) is unencrypted — acceptable when both run on the same container task
- Forward `X-Forwarded-For` and `X-Forwarded-Proto` headers for logging and OAuth flows

Phase needed: Phase 2 — request domain/subdomain allocation

2.5 DNS

What: Domain name resolution for the MCP server endpoint.

- **Records:** Single A/CNAME record pointing to the compute instance
- **Subdomain:** e.g. `mcp.{company-domain}` or `mcp-relay.{company-domain}`

Phase needed: Phase 2 — request subdomain allocation from infrastructure/network team

2.6 Logging & Monitoring

What: Centralised log aggregation and uptime monitoring for the server process.

- **Log format:** Structured JSON (already implemented in the server)
- **Log volume:** Low — one log entry per tool invocation + errors
- **Metrics:** Request count, latency, error rate per tool
- **Alerting:** Server down, sustained error rate, external API failures
- **Retention:** 30 days minimum

Phase needed: Phase 2 — request access to logging platform (e.g. CloudWatch, Datadog)

2.7 Container Registry

What: Storage for Docker images if using a containerised deployment.

- **Image size:** Small (~100MB — Node.js slim + compiled TypeScript)
- **Tagging:** Semantic versioning aligned with releases
- **Scanning:** Vulnerability scanning on push (desirable)

Phase needed: Phase 2 — request container registry access (e.g. ECR, ACR)

2.8 CI/CD Pipeline

What: Automated build, test, and deployment pipeline.

- **Triggers:** Push to main, PR checks
- **Steps:** Lint → format check → typecheck → test → build → deploy
- **Environments:** Staging → production (when team adoption warrants it)

- **Rollback:** One-click rollback to previous version

Phase needed: Phase 2 — confirm CI/CD platform (GitHub Actions likely sufficient if already in use)

2.9 Identity & Access Management

What: Authentication and authorisation for MCP clients connecting over HTTP.

- **Protocol:** OAuth 2.1 (MCP specification requirement for HTTP transport)
- **Provider:** Corporate identity provider (SSO) or standalone OAuth server
- **Scopes:** Per-tool or per-domain authorisation (future)

Phase needed: Phase 2 — request OAuth client registration or access to corporate IdP (longest lead time item)

---

3. Provider Options

For each infrastructure requirement, the table below lists viable providers. All options support Node.js workloads.

Requirement	AWS	Google Cloud Platform	Azure	Self-Hosted / Other
Compute	ECS Fargate, App Runner, EC2	Cloud Run, GKE, Compute Engine	Container Apps, App Service, AKS	Docker on VPS (Hetzner, DigitalOcean)
Secrets	Secrets Manager, Parameter Store	Secret Manager	Key Vault	HashiCorp Vault, Doppler
Object Storage	S3	Cloud Storage	Blob Storage	MinIO (S3-compatible)
Reverse Proxy	Caddy, Nginx (on Fargate/EC2)	Caddy, Nginx (on Cloud Run)	Caddy, Nginx (on Container Apps)	Caddy, Nginx, Traefik
DNS	Route 53	Cloud DNS	Azure DNS	Cloudflare, existing registrar
Logging	CloudWatch	Cloud Logging	Monitor / Log Analytics	Grafana + Loki, Datadog
Container Registry	ECR	Artifact Registry	ACR	Docker Hub, GitHub Container Registry
CI/CD	CodePipeline, CodeBuild	Cloud Build	Azure Pipelines	GitHub Actions, GitLab CI
Identity (OAuth)	Cognito	Identity Platform	Entra ID (Azure AD)	Auth0, Keycloak

---

4. Recommended Stack

Based on the project's current trajectory (small team, low traffic, cost-sensitive POC), the following is recommended as a starting point. This is a recommendation, not a commitment — the final choice depends on the team's existing cloud presence and preferences.

Primary Recommendation: AWS

AWS is recommended because the project already has an S3 dependency (brand guidelines) and Jira Cloud integration is region-agnostic.

Requirement	Service	Rationale
Compute	**ECS Fargate**	Serverless containers — no server management, scales to zero idle cost, supports long-running processes and SSE connections
Secrets	**Secrets Manager**	Native ECS integration (inject at task level), automatic rotation, audit via CloudTrail
Object Storage	**S3**	Already architected for brand guidelines; no change needed
Reverse Proxy	**Caddy**	Sidecar container in ECS task; automatic TLS via Let's Encrypt, SSE support, zero config. Load balancer deferred to Phase 3
DNS	**Route 53**	Or existing registrar with CNAME to compute instance
Logging	**CloudWatch Logs**	Native ECS log driver; structured JSON searchable
Container Registry	**ECR**	Private, integrated with ECS task definitions
CI/CD	**GitHub Actions**	Already using GitHub; deploy to ECS via `aws-actions/amazon-ecs-deploy-task-definition`
Identity	**Cognito**	OAuth 2.1 compliant; or integrate with existing corporate IdP

Alternative: Lightweight / Self-Hosted

For teams preferring simplicity or without AWS accounts:

Requirement	Service	Rationale
Compute	**Docker on VPS** (e.g. Hetzner, DigitalOcean)	Cheapest option; full control
Secrets	**Doppler** or **environment variables** on host	Simple secret injection
Object Storage	**S3-compatible** (e.g. DigitalOcean Spaces, MinIO)	Drop-in replacement via AWS SDK endpoint override
Reverse Proxy	**Caddy**	Automatic TLS via Let's Encrypt, zero config
CI/CD	**GitHub Actions** + SSH deploy	Straightforward for single-server setups
Identity	**Auth0** or **Keycloak**	Standalone OAuth provider

---

5. Phased Deployment

Phase 1 — Local MVP Development

No enterprise infrastructure required. Everything runs locally.

- stdio transport
- Jira tools (get, create, search, update)
- Brand guidelines tool (S3) — in progress, needs S3 bucket access
- Local `.env` credentials
- MCP Inspector for manual testing

Infrastructure action: Request S3 bucket access for brand guidelines (only Phase 1 dependency on enterprise services).

Phase 2 — Enterprise Access & Prototype Deployment

This is the critical phase. The code may be ready, but deployment is blocked without enterprise access.

Access Requests

Request	Why	Effort	Blocked Without It
OAuth client / IdP registration	MCP spec requires OAuth 2.1 for HTTP transport; need client ID + secret from corporate identity provider	High	Cannot authenticate team members
Container hosting (e.g. ECS Fargate, Cloud Run)	Need somewhere to run the server	Medium	Cannot deploy
Subdomain / DNS record	Teams need a URL to point their MCP clients at	Medium	Cannot connect
Secrets manager access	API tokens must not live in `.env` files on a shared server	Low	Cannot securely store credentials
Container registry (e.g. ECR, ACR)	Need to push Docker images	Low	Cannot deploy images
Logging platform access (e.g. CloudWatch, Datadog)	Need visibility into server health and errors	Low	Cannot monitor or debug
Network / firewall rules	Zscaler proxy in use; MCP servers currently work without whitelisting — escalate only if blocked	TBD	Potential blocker, not expected

Build Tasks (once access is granted)

- **Containerise** — create `Dockerfile` (Node.js slim, multi-stage build)
- **Spike S3** — validate Streamable HTTP transport with auth middleware
- **Implement OAuth 2.1** — integrate with corporate identity provider
- **Secrets migration** — move `.env` credentials into secrets manager
- **Provision compute** — deploy container to hosting service
- **Configure reverse proxy** — Caddy sidecar with TLS and SSE support
- **Configure DNS** — point subdomain to compute instance
- **Set up CI/CD** — automated build and deploy pipeline
- **Enable logging** — route structured JSON logs to centralised store
- **Configure monitoring** — health checks, error rate alerting
- **Onboard select teams** — provide MCP client configuration, gather feedback

Phase 3 — Scale & Harden

Once the prototype is validated with select teams, prepare for wider rollout.

- Evaluate horizontal scaling (multiple container instances, auto-scaling)
- Introduce load balancer (ALB or equivalent) to replace reverse proxy when scaling to multiple instances
- Add staging environment for testing new tool integrations
- Implement RBAC (per-team tool access, scoped permissions)

- Review cost optimisation (reserved capacity, spot instances)
- Harden monitoring and alerting (SLAs, on-call runbook)
- Expand tool catalogue (Phase 2+ tools from roadmap)

---

6. Cost Considerations

Phase 2 Estimated Monthly Cost (Prototype, AWS, Low Traffic)

Service	Estimate	Notes
ECS Fargate	£15–30	0.25 vCPU, 0.5GB RAM, business hours (includes Caddy sidecar)
ECR	<£1	Small images, low pull volume
Secrets Manager	<£5	~10 secrets, infrequent access
S3	<£1	Negligible for config files
CloudWatch	<£5	Low log volume
Route 53	<£1	Single hosted zone
Total	**~£20–40/month**	Conservative estimate

Cost Optimisation Levers

- **Scale to zero:** Fargate tasks can be stopped outside business hours via scheduled scaling
- **Spot capacity:** Fargate Spot for non-critical workloads (up to 70% savings)
- **Rightsizing:** Monitor actual resource usage and adjust task size
- **Reserved compute:** If running 24/7, consider Fargate Savings Plans

> **Note:** These estimates are indicative. Actual costs depend on usage patterns, region, and any existing AWS commitments (e.g. Enterprise Discount Programme).